

MedTracker: Comprehensive Deployment and Setup Guide

This document provides an exhaustive, step by step walkthrough for developers and users to download, configure, build, and host the MedTracker application. MedTracker is designed as an n-tier Django application, featuring a structured separation between the Data Layer (PostgreSQL), Business Layer (Python logic), Service Layer (API/Routing), and Presentation Layer (HTML/CSS Frontend).

This guide is written for everyone, from seasoned developers to those who have never touched a line of code or a database before.

1. Prerequisites and Environment Setup

Before starting, you must ensure your system has the right tools installed. These tools are the foundation upon which MedTracker runs.

Software & Platforms

- **Code IDE (The Editor):** [Visual Studio Code \(VS Code\)](#) is the primary development environment for this project. It is where you will view and edit the code.
- **Database Platform (The Storage):** [PostgreSQL](#) (Version 13 or higher recommended). This is the engine that stores all your medication and dose data.
- **Database Management (The UI for Data):** [Beekeeper Studio](#). This project was built using Beekeeper Studio because it is intuitive and clean for executing SQL scripts and inspecting your tables.
- **Runtime (The Engine):** [Python 3.10+](#). Python is the language the backend is written in. Ensure you check the box that says "Add Python to PATH" during installation.
- **Hosting Platform (The Cloud):** [Render](#). We use Render to host the application on the internet so it can be accessed from any device.

Required Dependencies

The project relies on specific versions of key libraries to ensure stability. These are listed in your requirements.txt file:

- asgiref==3.11.1: Handles asynchronous communication.
- Django==5.2.12: The main web framework.
- django-rest-framework==3.16.1: Powering the service layer and API.
- psycopg2-binary==2.9.11: The "bridge" that allows Python to talk to PostgreSQL.
- dj-database-url: A utility to help Django understand Render's database URLs.
- gunicorn: The production grade web server used for hosting on Render.

2. Getting the Project

Before you can run the code, you need to get it onto your local machine.

1. **Download from GitHub:** Navigate to the GitHub repository. Look for the bright green **Code** button near the top right. Click it and select **Download ZIP**.
2. **Unpack the Files:** Find the MedTracker.zip in your Downloads folder. Right click it and select **Extract All...** (Windows) or double click it (Mac). Move the extracted folder to a place you can easily find, like C:\Projects\MedTracker or your Desktop.
3. **Open in VS Code:** Launch **Visual Studio Code**. Go to the top menu, click File > Open Folder, and select the folder you just unpacked. You should now see the project structure (including the backend folder and requirements.txt) in the left sidebar.

3. Local Deployment (Development)

This section covers how to get MedTracker running on your own computer for testing or personal use.

A. Setting up the Data Layer (PostgreSQL)

If you do not have PostgreSQL installed, follow these detailed steps to prepare your database environment:

1. **Installation:** Download the PostgreSQL installer from the [EnterpriseDB website](#). Run the .exe or .dmg installer.
 - **CRITICAL:** During the setup, it will ask you to create a password for the "postgres" user. **Write this down.** You will need it to connect your app to the database.
2. **Verify the Service:** Ensure PostgreSQL is running. On Windows, you can check the "Services" app to see if "postgresql-x64" is "Running."
3. **Beekeeper Studio Connection:**
 - Open **Beekeeper Studio**.
 - Click **New Connection** and choose **Postgres**.
 - **Host:** localhost
 - **Port:** 5432
 - **User:** postgres
 - **Password:** The password you wrote down in step 1.
 - Click **Test Connection** to make sure it works, then click **Connect**.
4. **Create the Database:**
 - Once connected, you will see a list of databases (usually just "postgres").
 - Right click the connection name in the sidebar and select **Create Database**.
 - Type mabel as the name and click **Create**.
5. **Initialize Schema & Data:**
 - Ensure the mabel database is selected. Open a new **Query** tab.
 - Type and run: CREATE EXTENSION IF NOT EXISTS "pgcrypto"; (This is vital because we use UUIDs for security).
 - In VS Code, open create_db.sql. Copy every line of code.
 - Go back to Beekeeper, paste the code into the query tab, and click **Run**.
 - Now, open fill_db.sql in VS Code, copy its contents, paste them into a fresh query tab in Beekeeper, and click **Run**. Your Data Layer is now ready.

B. Configuring the Service Layer

1. In VS Code, navigate to the file backend/medtracker/settings.py.
2. Use Ctrl + F to search for DATABASES.
3. Ensure your local block looks exactly like this, replacing the password with your actual Postgres password:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'mabel',  
        'USER': 'postgres',  
        'PASSWORD': 'your_actual_password_here',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

C. Building the Application Environment

1. Open the VS Code Terminal by pressing Ctrl + ` (the backtick key).
2. **Create a Virtual Environment:** This keeps the project dependencies separate from your computer, which is a best practice.
 - Run: `python -m venv venv`
3. **Activate the Environment:**
 - **Windows:** `.\venv\Scripts\activate`
 - **Mac/Linux:** `source venv/bin/activate`
 - *Note: You should now see (venv) in parenthesis at the start of your terminal line.*
4. **Install Dependencies:** This downloads all the necessary libraries required for the project to function.
 - Run: `pip install -r requirements.txt`

D. Running the Project

1. **Navigate to the Backend:** You must be in the same folder as the manage.py file to run Django commands.
 - Run: `cd backend`
2. **Start the Server:**
 - Run: `python manage.py runserver`
3. **Identify Success:** Your terminal will say "Starting development server at `https://www.google.com/search?q=http://127.0.0.1:8000/`".
 - Open your web browser and go to `http://127.0.0.1:8000`.
 - If you see the MedTracker dashboard and see a list of medications (the Presentation Layer), you have successfully deployed locally!

4. Cloud Deployment (Production via Render)

Ready to show the world? Here is how to host it on Render.

A. Hosting the Data Layer (Managed Postgres)

1. **Create the Database:** On your Render Dashboard, click **New +** and select **PostgreSQL**.
2. **Configuration:** Name it medtracker-db. Choose the "Free" tier for testing. Click **Create Database**.
3. **Retrieve Connection String:** Once the database status turns green and says "Available," look for the **External Connection String**. It looks like a long URL starting with postgres://. Copy this.
4. **Seeding Production:**
 - Open **Beekeeper Studio**.
 - Create a **New Connection**, select **Postgres**, and paste that long URL into the "Import from URL" field.
 - Once connected to your Render cloud database, run the pgcrypto extension command, the create_db.sql script, and the fill_db.sql script, just like you did for the local setup.

B. Configuring the Code for Production

1. In VS Code, open backend/medtracker/settings.py.
2. **Switch the Database Logic:**
 - Find your local DATABASES block and add a # to the start of every line to "comment it out."
 - Find the Render block and remove the # symbols so it looks like this:

UNCOMMENT THIS LINE TO DEPLOY ON RENDER

```
DATABASES = {
    'default': dj_database_url.config(
        # This pulls the DATABASE_URL environment variable from Render
        default=os.environ.get('DATABASE_URL'),
        conn_max_age=0,
        conn_health_checks=True,
    )
}
```

Add the search path logic so your 'medtracker' schema is recognized

```
DATABASES['default']['OPTIONS'] = {
    'options': '-c search_path=medtracker,public'
}
```

3. **Save and Push:** Save the file, commit your changes, and push them to your GitHub repository.

C. Hosting the Web & Presentation Layers

1. **Create Web Service:** In Render, click **New + > Web Service**.
2. **Connect Repo:** Select your GitHub repository.
3. **Configuration Settings:**
 - **Name:** medtracker-app
 - **Runtime:** Python 3
 - **Build Command:** pip install -r requirements.txt
 - **Start Command:** gunicorn --chdir backend medtracker.wsgi
 - *Note: The --chdir backend is crucial because the "root" of your repo is not where the Django app lives, the app lives inside the backend folder.*
4. **Advanced Environment Variables:** Click the "Advanced" button and add these variables:
 - `DATABASE_URL`: Paste your **Internal Connection String** from your Render Database page here.
 - `PYTHON_VERSION`: 3.10.0
 - `SECRET_KEY`: Create a random string of characters here to secure your application.

5. Final Verification

Wait for the Render log to say "Your service is live." Click the URL provided at the top of the Render page (usually something like <https://medtracker-app.onrender.com>).

If you see your app and your data, congratulations! You have successfully navigated the entire n-tier deployment process from "Download ZIP" to a fully hosted cloud application. "cool, that works!"